这张图片描述的是 **Tarjan 算法** 用于解决 **LCA（最近公共祖先，Lowest Common Ancestor）** 问题的过程。

Tarjan 算法是一个非常有名的**离线算法**。所谓“离线”，是指它必须先读入所有的查询请求，然后统一进行处理，一次性给回所有答案，而不是像在线算法那样问一个答一个。

下面我为你详细解释一下图片中的五个步骤及其背后的逻辑：

核心思想

Tarjan 算法结合了 **DFS（深度优先搜索）** 和 **并查集（Disjoint Set Union, DSU）**。
在 DFS 过程中，当我们访问到一个节点并处理完它的所有子树后，利用并查集将该节点指向它的父亲。这样，如果我们查询两个节点 (u, v) 的 LCA，而此时 u 正在被访问，v 已经访问过并处理完了，那么 v 在并查集中的根节点就是 u 和 v 的最近公共祖先。

详细步骤解读

1. 预处理查询边 (Offline Processing)

- **解释：** 因为是离线算法，我们需要把所有的查询 (u, v) 存起来。为了方便在 DFS 到节点 u 时能立刻找到所有与 u 有关的查询，我们将查询也像邻接表一样存起来。
- **关键点：** 查询 (u, v) 要存两次，即存一个 $u \to v$ 的查询，也存一个 $v \to u$ 的查询，因为你不知道 DFS 会先扫到哪一个。

2. DFS 遍历与状态记录

- **解释：** 从根节点开始进行深度优先搜索。
- **`visited` 数组：** 标记节点状态（未访问、正在访问、已访问完）。
- **`parent` 数组：** 虽然图片里写记录父亲，但在并查集的逻辑里，它主要用于实现 `find` 操作。

3. 回溯思想与并查集更新

- **这是最核心的一步：**
  - 当我们刚进入节点 u 时，在并查集中，我们认为 u 的祖先就是它自己（`fa[u] = u`）。
  - 接着递归访问 u 的所有子节点。
  - **重点：** 当一个子节点 v 的所有后代都处理完了，准备回退回 u 时，我们将 v 的并查集根节点指向 u（`fa[v] = u`）。
  - 这意味着：**一个节点的并查集指向，代表了它向上走能遇到的“最先被处理完”的祖先。**

4. 在回溯时更新结果

- **解释：** 当我们正在处理节点 u 时，检查所有与 u 相关的查询 (u, v)。
- **判断条件：** 如果 v 已经被访问过且已经回溯过了（即标记为已处理），那么此时 `find(v)` 返回的并查集根节点，就是 u 和 v 的 LCA。
- **直观理解：** 想象 DFS 的过程，当你在 u 点，而 v 已经处理完了，通过并查集的路径压缩，从 v 一路往上找，第一个还没处理完（或者说正在处理中）的共同祖先就是它们的 LCA。

5. 输出结果

- 将所有查询的结果按照最初读入的顺序输出即可。

总结

- **优点：** 效率极高。时间复杂度接近 $O(n + m)$（n 为节点数，m 为查询数），其中并查集操作几乎是常数级。
- **缺点：** 必须预知所有查询（离线），且不支持动态修改树的结构。

简单来说： Tarjan 算法就像是一个人在树上做 DFS 旅游，每走完一个景点（子树）就把这个景点挂靠到它的上一级（父亲）。当他走到一个新景点 u 时，如果发现之前去过的景点 v 和 u 有“合影需求”（查询），他只需要看 v 现在挂靠在哪个大老板（并查集根节点）名下，那个老板就是两人的 LCA。